

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Ingeniería en Ciencias y Sistemas
CURSO: *Manejo e Implementación de Archivos*



MANUAL TÉCNICO

ExtreamFS — Simulador de Sistema de Archivos EXT2

Nombre:

ADILZON ALFREDO VELÁSQUEZ HERNÁNDEZ

Carné No.:

201908076

Auxiliar: Williams Bramlley Constanza Oscal

Guatemala, 13 de Septiembre 2026

1. Arquitectura del sistema

ExtreamFS está compuesto por tres capas claramente separadas, tal como lo exige el enunciado del proyecto: un backend obligatorio en C++ que implementa un sistema de archivos EXT2 simulado sobre archivos binarios (.mia), una API RESTful que expone ese backend a través de HTTP, y un frontend en React que permite ejecutar comandos y visualizar los reportes generados. A diferencia de un simple analizador léxico, el backend ejecuta las operaciones reales: crea y escribe discos binarios, administra particiones, formatea EXT2, gestiona usuarios y permisos, crea archivos y carpetas, y genera reportes con Graphviz.

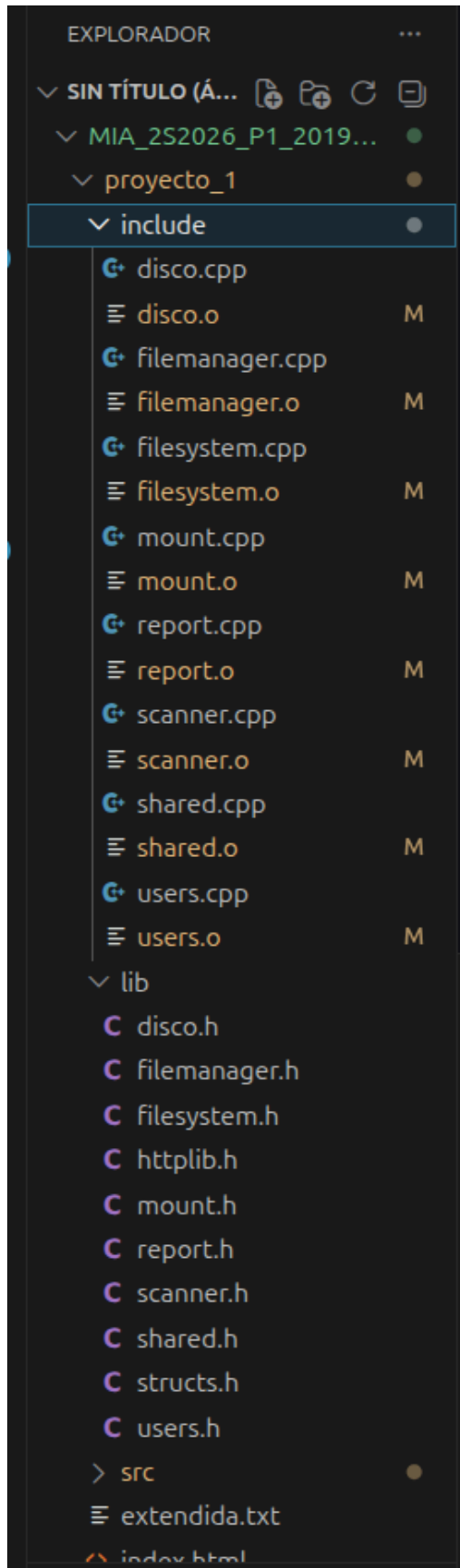
1.1 Diagrama de arquitectura

Capa	Componente	Responsabilidad
Frontend	App.jsx (React + Vite)	Área de entrada/salida de comandos, carga de scripts .smia, envío de peticiones HTTP y visualización de reportes (imágenes/PDF/texto).
Comunicación	API REST (JSON / texto plano)	POST /api/execute ejecuta uno o varios comandos y devuelve la salida en texto. GET /api/file sirve los reportes generados en disco.
Backend — API	api_main.cpp + httpplib.h	Levanta el servidor HTTP, gestiona CORS, delega la ejecución al Scanner y sirve archivos de reportes.
Backend — Núcleo	scanner, Disk, Mount, FileSystem, Users, FileManager, Report	Implementan la lógica real de discos, particiones, EXT2, usuarios/permisos, archivos/carpetas y reportes.
Persistencia	Archivos .mia	Simulan discos duros físicos mediante archivos binarios de tamaño fijo, donde se escriben MBR, EBR, Superbloque, bitmaps, inodos y bloques.

1.2 Flujo de una petición

Paso	Descripción
1	El usuario escribe uno o varios comandos en el área de entrada del frontend, o carga un archivo .smia.
2	Al presionar "Ejecutar", el frontend envía todo el bloque de texto como POST a /api/execute (parámetro opcional confirm=true/false para operaciones que requieren confirmación, como RMDISK o sobrescritura de archivos).
3	El backend recibe la petición; scanner::executeScript() separa el texto en líneas y ejecuta cada comando con scanner::execute().
4	Cada comando invoca la clase correspondiente (Disk, Mount, FileSystem, Users o FileManager), la cual realiza la operación real sobre el archivo .mia.
5	La salida de cada operación (impresa originalmente con cout) se captura en un stringstream, se limpia de códigos de color ANSI, y se concatena.
6	El backend responde con el texto plano acumulado; el frontend lo muestra en el área de salida.
7	Si el comando fue REP, el usuario puede pegar la ruta del reporte generado en el visor de reportes, que solicita el archivo vía GET /api/file y lo renderiza como imagen, PDF o texto según su extensión.

1.3 Estructura de carpetas del proyecto



1.4 Modo API vs. modo consola

El backend puede ejecutarse en dos modalidades sin duplicar lógica: la consola original (src/main.cpp) y el servidor HTTP (src/api_main.cpp). Ambas comparten exactamente las mismas clases e instancias de negocio; lo único que cambia es cómo se recibe el comando y cómo se entrega la salida. Una bandera global (API_MODE) evita que operaciones que normalmente piden confirmación por teclado (como eliminar un disco en RMDISK, o sobrescribir un archivo existente en MKFILE) bloqueen el hilo del servidor: en modo API, la confirmación se resuelve mediante el parámetro confirm de la petición HTTP.

2. Estructuras utilizadas

Todas las estructuras que representan el disco físico y el sistema de archivos EXT2 están centralizadas en structs.h y se usan de forma idéntica en la creación, lectura y en los reportes, garantizando que lo que se escribe en el .mia sea exactamente lo que luego se interpreta.

2.1 Master Boot Record (MBR)

Campo	Tipo	Descripción
mbr_tamano	int	Tamaño total del disco en bytes.
mbr_fecha_creacion	time_t	Fecha y hora de creación del disco.
mbr_disk_signature	int	Número aleatorio que identifica de forma única al disco.
disk_fit	char	Ajuste usado para crear particiones: B (Best), F (First) o W (Worst).
mbr_Partition_1..4	Partition	Las 4 particiones posibles del disco (primarias o una extendida).

2.2 Partition

Campo	Tipo	Descripción
part_status	char	'0' libre / '1' en uso.
part_type	char	P (primaria), E (extendida) o L (lógica, solo dentro de una extendida).
part_fit	char	Ajuste de la partición: B, F o W.
part_start	int	Byte del disco donde inicia la partición.
part_size	int	Tamaño total de la partición en bytes.
part_name	char[16]	Nombre de la partición.
part_correlative	int	-1 hasta que se monta; luego indica el número de partición montada dentro de su disco.
part_id	char[4]	ID asignado al montar, ej. "341A" (carne + correlativo + letra).

2.3 Extended Boot Record (EBR)

Campo	Tipo	Descripción
part_status	char	'0' libre / '1' en uso.
part_fit	char	Ajuste de la partición lógica.
part_start	int	Byte donde inicia la partición lógica.
part_size	int	Tamaño de la partición lógica.
part_next	int	Byte del siguiente EBR; -1 si no hay más.
part_name	char[16]	Nombre de la partición lógica.

2.4 Superblock

Campo	Tipo	Descripción
-------	------	-------------

s_filesystem_type	int	Identificador del sistema de archivos (2 = EXT2).
s_inodes_count / s_blocks_count	int	Cantidad total de inodos / bloques de la partición.
s_free_inodes_count / s_free_blocks_count	int	Cantidad de inodos / bloques libres. Se actualiza en cada operación que asigna o libera.
s_mtime / s_umtime	time_t	Última fecha de montaje / desmontaje.
s_mnt_count	int	Veces que se ha montado el sistema.
s_magic	int	Valor fijo 0xEF53 que identifica EXT2.
s_inode_size / s_block_size	int	sizeof(Inodes) / sizeof(Folderblock).
s_first_ino / s_first_blo	int	Primer inodo / bloque libre conocido.
s_bm_inode_start / s_bm_block_start	int	Inicio del bitmap de inodos / bloques.
s_inode_start / s_block_start	int	Inicio de la tabla de inodos / bloques.

Distribución física de una partición formateada (en el orden exacto en que se escribe en el .mia):

Superbloque Bitmap Inodos Bitmap Bloques Tabla de Inodos Tabla de Bloques

El número de inodos (n) y de bloques (3n) se calcula despejando n de:

```
tamaño_particion = sizeof(Superblock) + n + 3n + n*sizeof(Inodes) + 3n*sizeof(Fileblock)
```

2.5 Inodes

Campo	Tipo	Descripción
i_uid / i_gid	int	Propietario y grupo propietario (usuario que estaba logueado al crearlo).
i_size	int	Tamaño real del contenido en bytes.
i_atime / i_ctime / i_mtime	time_t	Fechas de último acceso, creación y modificación.
i_block[15]	int	12 apuntadores directos (0–11) + 1 indirecto simple (12) + 2 reservados para doble/triple indirecto (13, 14, no implementados en este alcance).
i_type	char	0 = carpeta, 1 = archivo.
i_perm[3]	char	Permisos UGO en octal, un carácter por categoría (Usuario, Grupo, Otro), ej. {'6','6','4'}.

2.6 Bloques

Estructura	Campos	Uso
Content	b_name[12], b_inodo	Una entrada dentro de un bloque de carpeta: nombre + inodo al que apunta.
Folderblock	b_content[4] (Content)	Bloque de carpeta: hasta 4 entradas (archivos o subcarpetas).
Fileblock	b_content[64] (char)	Bloque de archivo: hasta 64 bytes de contenido.
Pointerblock	b_pointers[16] (int)	Bloque de apuntadores indirectos: 16 punteros a otros bloques de datos.

Cuando el contenido de una carpeta o archivo supera los 12 bloques directos disponibles (768 bytes), el sistema asigna automáticamente un bloque de apuntadores en i_block[12] y continúa referenciando bloques adicionales a través de él. El doble y triple indirecto (i_block[13] e i_block[14]) quedan fuera del alcance implementado en este proyecto, dado que no existen comandos de eliminación que exijan reorganizar árboles muy profundos.

3. Comandos implementados

A diferencia de un analizador que solo valida sintaxis, cada uno de los siguientes 16 comandos ejecuta la operación real sobre el archivo .mia correspondiente: escribe estructuras binarias, asigna inodos y bloques marcando los bitmaps, y actualiza los contadores del superbloque.

3.1 Administración de discos

Comando	Parámetros obligatorios	Parámetros opcionales	Función
MKDISK	-size, -path	-fit (BF/FF/WF, def. FF), -unit (K/M, def. M)	Crea un archivo .mia lleno de ceros binarios que simula un disco duro.
RMDISK	-path	—	Elimina el archivo .mia, previa confirmación.
FDISK	-size, -path, -name	-unit (B/K/M), -type (P/E/L), -fit (BF/FF/WF)	Crea una partición primaria, extendida o lógica dentro del disco.

3.2 Montaje

Comando	Parámetros	Función
MOUNT	-path, -name	Monta una partición primaria en memoria (RAM). Genera un ID único (2 últimos dígitos del carné + correlativo de partición + letra de disco) y actualiza part_status/part_correlative/part_id en memoria.
MOUNTED	sin parámetros	Lista todas las particiones actualmente montadas y sus IDs.

3.3 Sistema de archivos

Comando	Parámetros	Función
MKFS	-id (obligatorio), -type (opcional, Full)	Formatea la partición montada como EXT2: escribe el superbloque, los bitmaps, reserva la tabla de inodos/bloques, y crea la carpeta raíz junto con el archivo users.txt (con el usuario root inicial).

3.4 Usuarios, grupos y sesión

Comando	Parámetros	Función
LOGIN	-user, -pass, -id	Inicia sesión validando contra users.txt de la partición indicada.
LOGOUT	sin parámetros	Cierra la sesión activa.
MKGRP / RMGRP	-name	Crea / elimina un grupo (solo root).
MKUSR	-user, -pass, -grp	Crea un usuario, validando que el grupo exista (solo root).
RMUSR	-user	Elimina un usuario (solo root).
CHGRP	-user, -grp	Cambia el grupo de un usuario existente (solo root).

El archivo users.txt se administra como un archivo real dentro del sistema de archivos simulado: crece dinámicamente asignando bloques adicionales (directos e indirectos) conforme se agregan usuarios y grupos, en vez de estar limitado a un único bloque de 64 bytes.

3.5 Archivos y carpetas

Comando	Parámetros	Función
MKDIR	-path; opcional -p	Crea una carpeta. Con -p crea las carpetas padre faltantes. Valida permiso de escritura en el padre y rechaza duplicados.

MKFILE	-path; opcionales -r, -size, -cont	Crea un archivo. -size rellena con dígitos 0-9 cíclicos; -cont carga contenido desde un archivo del sistema operativo (tiene prioridad sobre -size). Con -r crea carpetas padre faltantes.
CAT	-file1..fileN	Muestra el contenido de uno o varios archivos concatenados, validando permiso de lectura.

El propietario (i_uid/i_gid) de cada archivo o carpeta creado es siempre el usuario realmente logueado en la sesión, y toda escritura valida los permisos UGO (Usuario/Grupo/Otro) del padre antes de proceder, salvo para el usuario root, que tiene siempre acceso total (777).

3.6 Reportes (REP)

El comando REP recibe -name, -path (ruta y extensión de salida) e -id; -path_file_ls es adicional para los reportes file y ls.

Nombre	Contenido	Formato de salida
mbr	Información del MBR y de los EBR de particiones lógicas.	Imagen/PDF (Graphviz)
disk	Distribución de particiones y espacio libre como porcentaje del disco.	Imagen/PDF (Graphviz)
inode	Todos los inodos en uso, con sus 15 apuntadores.	Imagen/PDF (Graphviz)
block	Todos los bloques en uso (carpeta o archivo).	Imagen/PDF (Graphviz)
bm_inode / bm_block	Bitmap completo de inodos / bloques, 20 registros por línea.	Texto plano (.txt)
sb	Todos los campos del superbloque.	Imagen/PDF (Graphviz)
tree	Árbol completo del sistema de archivos, inodos y bloques enlazados.	Imagen/PDF (Graphviz)
file	Contenido íntegro del archivo indicado en -path file ls.	Texto plano
ls	Listado de una carpeta (o un archivo puntual) con permisos, propietario, grupo, tamaño, fecha y tipo.	Imagen/PDF (Graphviz)

La extensión del archivo de salida (-path) determina el formato de renderizado: el backend invoca Graphviz con "-T<extensión>", de modo que el mismo comando puede producir .jpg, .png o .pdf según lo que el usuario indique.

4. Ejemplos de uso end-to-end

La siguiente secuencia fue ejecutada y verificada contra el sistema en funcionamiento, cubriendo el flujo completo: creación de disco, partición, montaje, formateo, sesión, usuarios, archivos y reporte.

```
mkdisk -size=10 -unit=M -path=/home/user/Disco1.mia
Válido - disco de 10 MB creado, ajuste FF por defecto.

fdisk -size=5 -unit=M -path=/home/user/Disco1.mia -name=Part1
Válido - partición primaria creada.

mount -path=/home/user/Disco1.mia -name=Part1
Válido - devuelve id=761A (ejemplo con carné terminado en 76).

mkfs -id=761A
Válido - partición formateada como EXT2 (carpeta raíz + users.txt).

login -user=root -pass=123 -id=761A
Válido - sesión iniciada como root.
```

```

mkgrp -name=usuarios
mkusr -user=user1 -pass=usuario -grp=usuarios
    Válido – ambos validados contra users.txt.

mkdir -path=/home/docs -p
mkfile -size=800 -path=/home/docs/a.txt
    Válido – 800 bytes fuerzan el uso de bloque indirecto (>12 bloques directos).

cat -file1=/home/docs/a.txt
    Muestra el contenido cíclico 0-9 de 800 bytes.

rep -id=761A -path=/home/user/reports/tree.pdf -name=tree
    Válido – genera el árbol completo del sistema de archivos en PDF.

```

4.1 Casos de error verificados

Entrada	Resultado
mkdisk -size=-5 -path=/home/x.mia	Error: size debe ser mayor a 0 (no se crea el archivo).
mkdisk -size=10 -unit=G -path=/home/x.mia	Error: valor de -unit no válido (debe ser K o M).
mount -path=/home/Disco1.mia -name=Ext1 (partición extendida)	Error: no se permite montar una partición extendida.
mkusr -user=user2 -pass=x -grp=noexiste	Error: el grupo indicado no existe.
mkdir -path=/home (ya existente)	Error: el directorio ya existe.
mkfile sobre archivo existente sin confirmar	Operación cancelada (requiere confirm=true en modo API).
rep -id=761A -path=/home/x.jpg -name=journaling	Error: no es un reporte válido (fuera del alcance EXT2 de este proyecto).

5. Diseño interno y diagramas técnicos

Esta sección complementa la descripción funcional con una vista interna del sistema, respetando las estructuras, flujo y alcance descritos en este manual.

5.1 Arquitectura de 3 capas

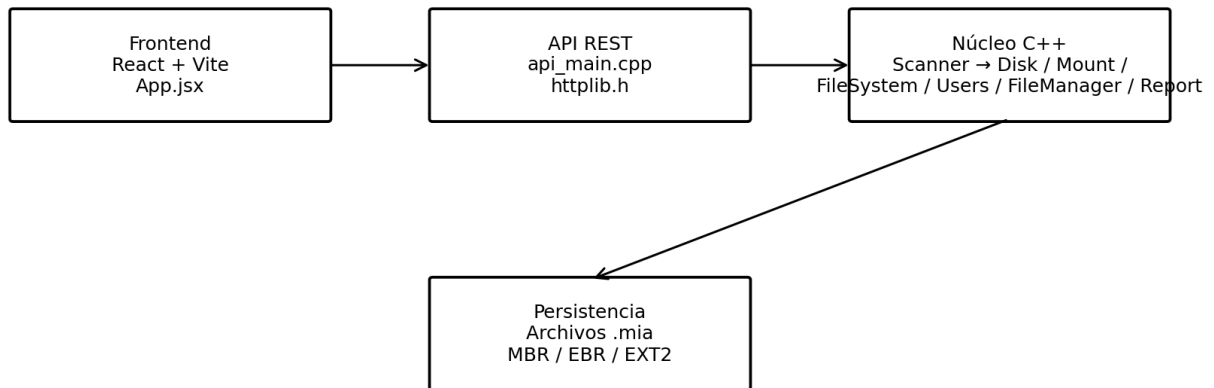


Figura 1. Arquitectura lógica de ExtreamFS.

React/Vite constituye la capa de presentación; api_main.cpp expone el backend por HTTP; el núcleo C++ contiene Scanner y las clases de negocio. La persistencia se realiza sobre archivos .mia.

5.2 Diagrama de clases

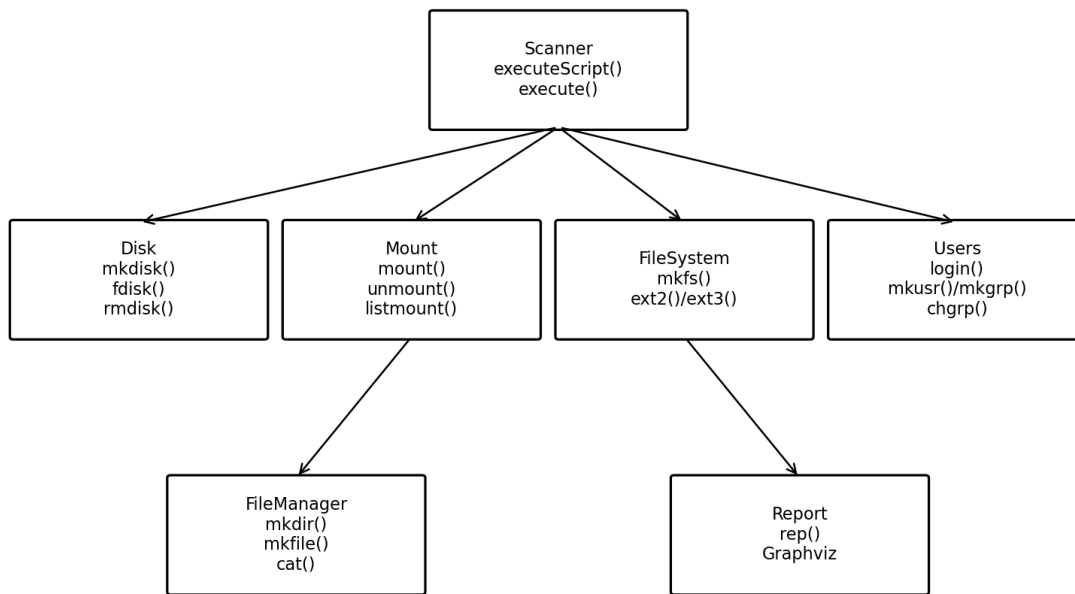


Figura 2. Clases principales y delegación.

Scanner recibe y despacha comandos hacia Disk, Mount, FileSystem, Users, FileManager y Report. La documentación no registra herencia entre estas clases; la relación principal es de delegación y separación de responsabilidades.

5.3 Flujo de una petición

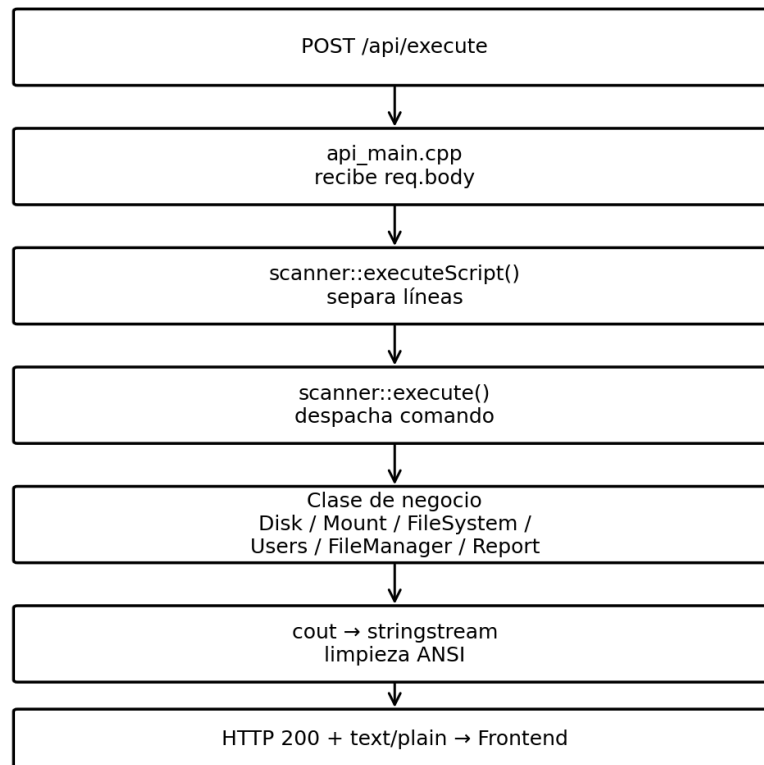
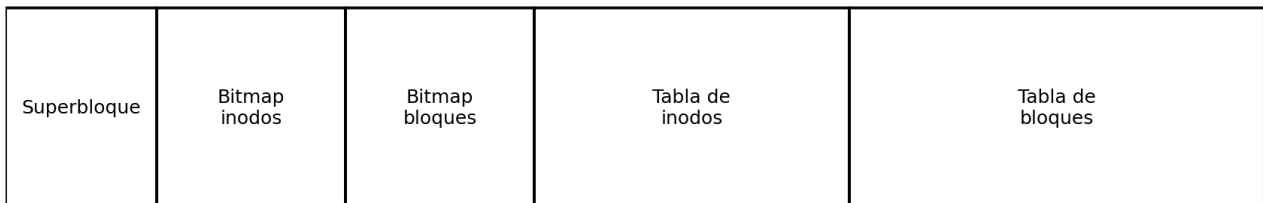


Figura 3. *POST → Scanner → clase → salida capturada → respuesta HTTP.*

El frontend envía el script mediante POST /api/execute. executeScript separa líneas, execute despacha y la salida producida por cout se captura, limpia y devuelve como text/plain.

5.4 Distribución física EXT2

Distribución física documentada de la partición EXT2



Offsets determinados por los campos del Superblock

Figura 4. *Orden físico de la partición.*

El orden documentado es: Superblock → Bitmap de inodos → Bitmap de bloques → Tabla de inodos → Tabla de bloques. Los offsets concretos se almacenan en el Superblock.

5.5 Árbol de directorios

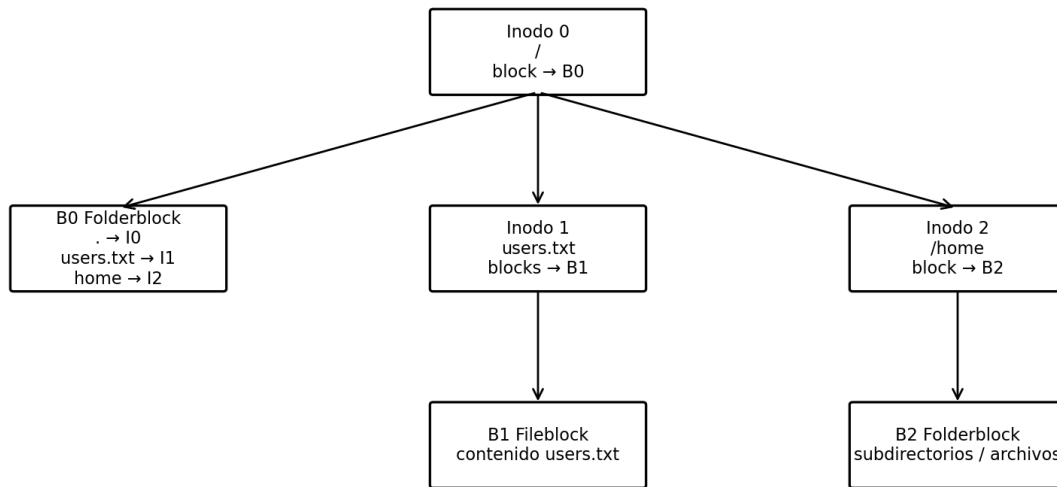


Figura 5. Relación conceptual entre inodos y bloques.

Los Folderblock contienen entradas Content que relacionan nombres con números de inodo. Los inodos apuntan posteriormente a los bloques de contenido.

5.6 Bloque de apuntes

Pointerblock: 16 punteros a bloques de datos

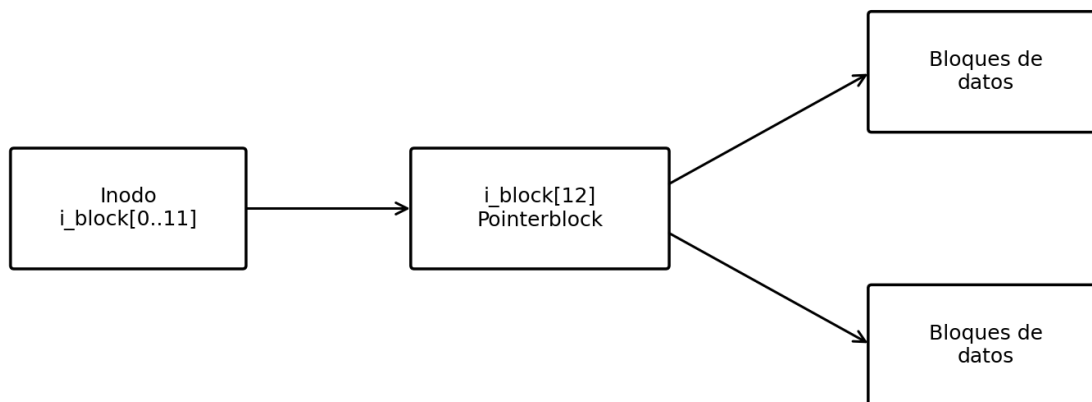


Figura 6. i_block[12] y direccionamiento indirecto simple.

Cuando los 12 apuntes directos no son suficientes, i_block[12] referencia un Pointerblock con 16 punteros a bloques de datos. i_block[13] e i_block[14] quedan reservados.

6. Detalles de implementación

6.1 Cálculo de inodos y bloques

La fórmula documentada es:

$$\text{tamaño_particion} = \text{sizeof}(\text{Superblock}) + n + 3n + n \cdot \text{sizeof}(\text{Inodes}) + 3n \cdot \text{sizeof}(\text{Fileblock})$$

Ejemplo: para 10 MiB (10,485,760 bytes), usando los tamaños típicos de las estructuras descritas en una compilación de 64 bits —Superblock $\approx$ 76 bytes, Inodes $\approx$ 104 bytes y Fileblock = 64 bytes— se obtiene $n = \text{floor}((10,485,760 - 76) / 300) = 34,952$. Se reservan 34,952 inodos y 104,856 bloques. El resultado debe comprobarse con `sizeof()` en el entorno real si cambia el layout de las estructuras.

6.2 Manejo de bitmaps

Cada posición del bitmap representa un inodo o bloque. Al asignarlo se marca como ocupado y se actualiza el contador libre del Superblock; al liberarlo se marca como disponible y se incrementa el contador. Para localizar un recurso libre se recorre el bitmap hasta encontrar la primera posición disponible.

6.3 Manejo de bloques indirectos

Los bloques 0–11 del inodo son directos. Al superar 12 bloques se reserva un Pointerblock en `i_block[12]`. Cada Pointerblock contiene 16 enteros. Con bloques de 64 bytes, el alcance máximo documentado es $12 + 16 = 28$ bloques de datos, equivalentes a 1,792 bytes.

6.4 users.txt

`users.txt` es un archivo real dentro de la partición. Su inodo apunta a bloques de contenido; el sistema lee el texto, separa sus registros, valida grupos/usuarios y vuelve a escribirlo. Al crecer, se asignan bloques adicionales y se usa `i_block[12]` cuando ya no bastan los directos.

6.5 Permisos UGO

`i_perm[3]` guarda un carácter para Usuario, Grupo y Otro. Las operaciones de escritura validan primero el permiso del propietario, luego el grupo y finalmente Otro. El propietario y grupo se obtienen de `i_uid/i_gid`. Root tiene acceso total (777) según el alcance documentado.

6.6 API_MODE y confirmaciones

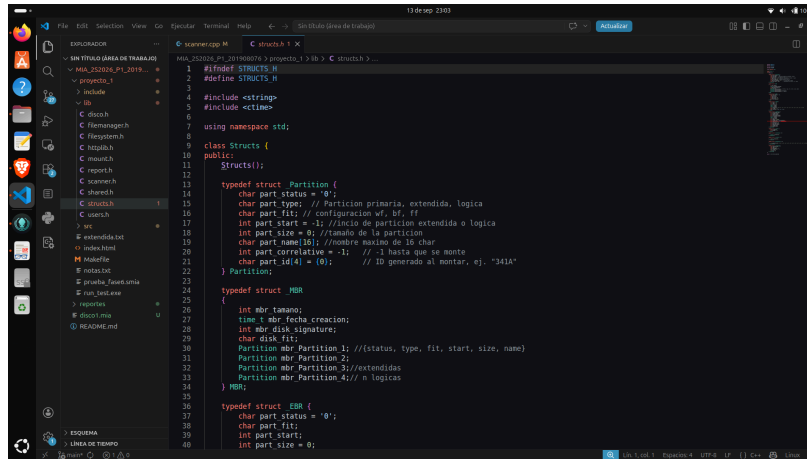
`API_MODE` evita que operaciones que normalmente esperan entrada por teclado bloqueen el servidor. La confirmación se transporta mediante el parámetro `confirm` de la petición HTTP.

6.7 Captura de cout, CORS y ANSI

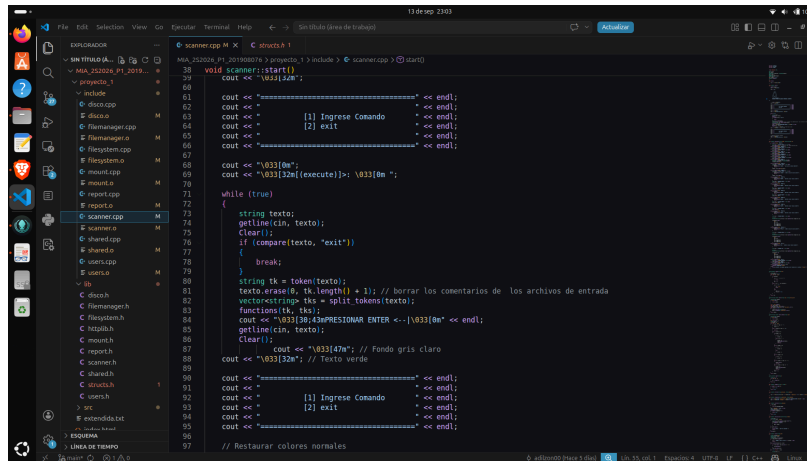
La salida del backend se redirige temporalmente a un stringstream para convertirla en texto HTTP. La API configura headers CORS para permitir la comunicación con el frontend. Finalmente se eliminan códigos ANSI para que las secuencias de color de consola no aparezcan como caracteres de control en el navegador.

7. Fragmentos de código clave

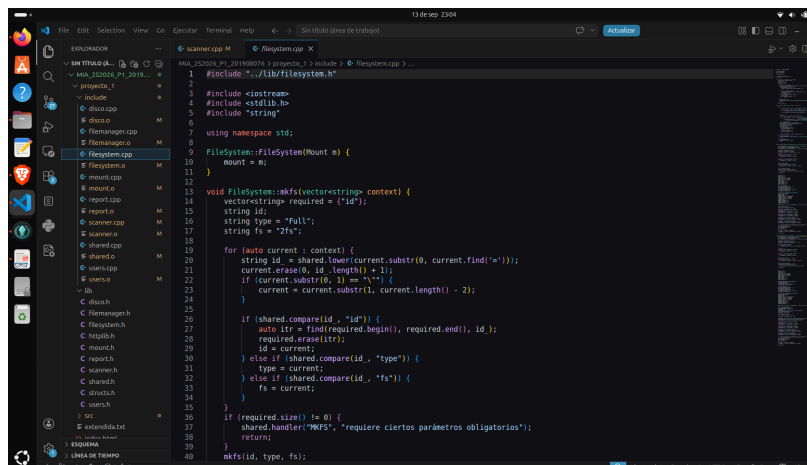
7.1 structs.h — estructuras EXT2



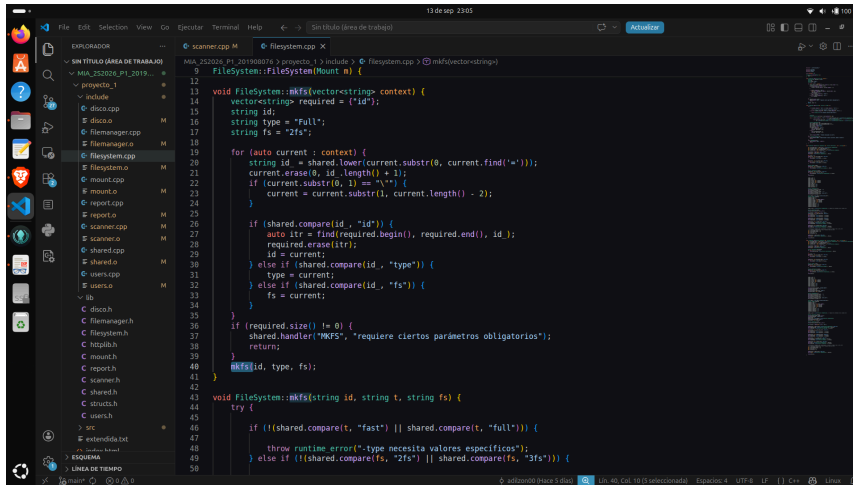
7.2 scanner.cpp — executeScript() y execute()



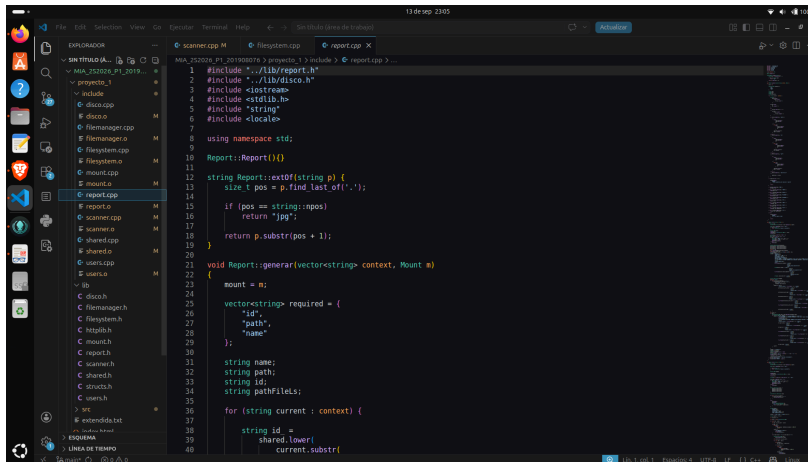
7.3 filesystem.cpp — cálculo de n



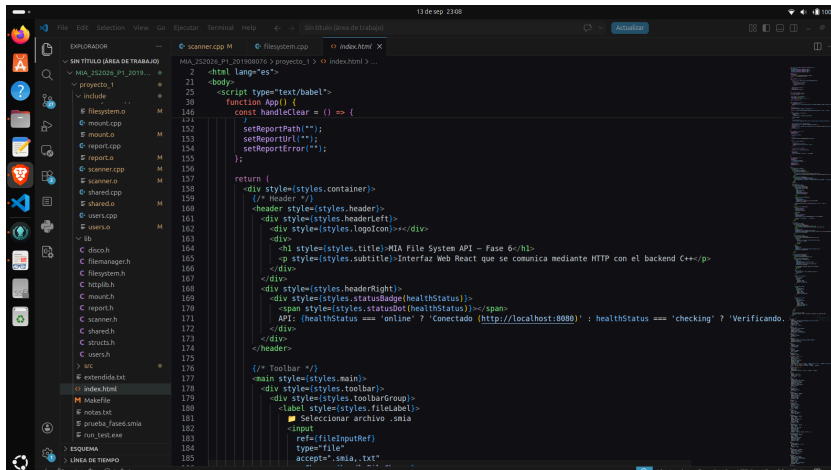
7.4 filesystem.cpp — mkfs()



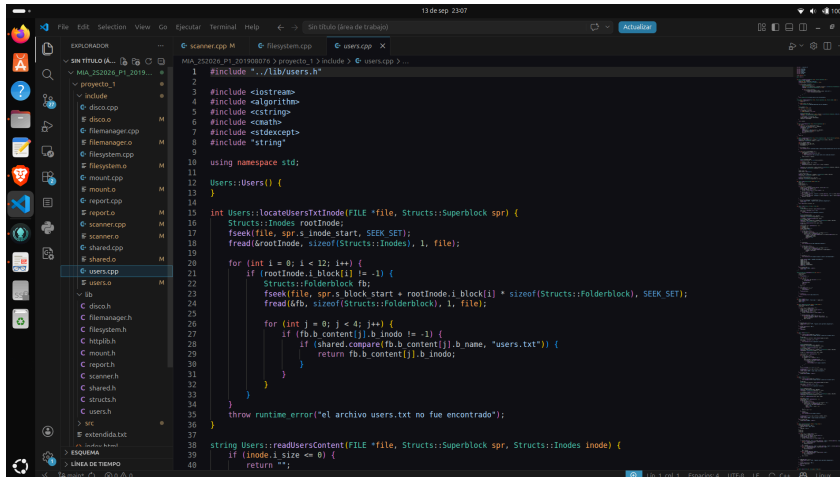
7.5 report.cpp — Graphviz



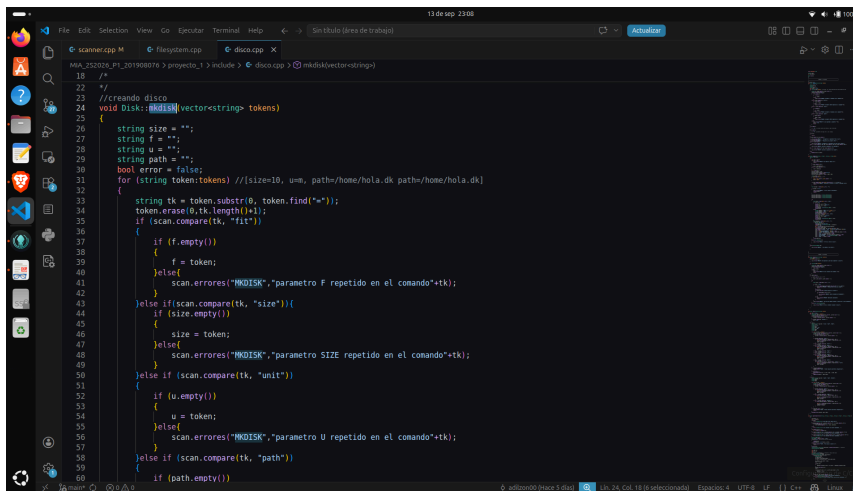
7.6 index.html



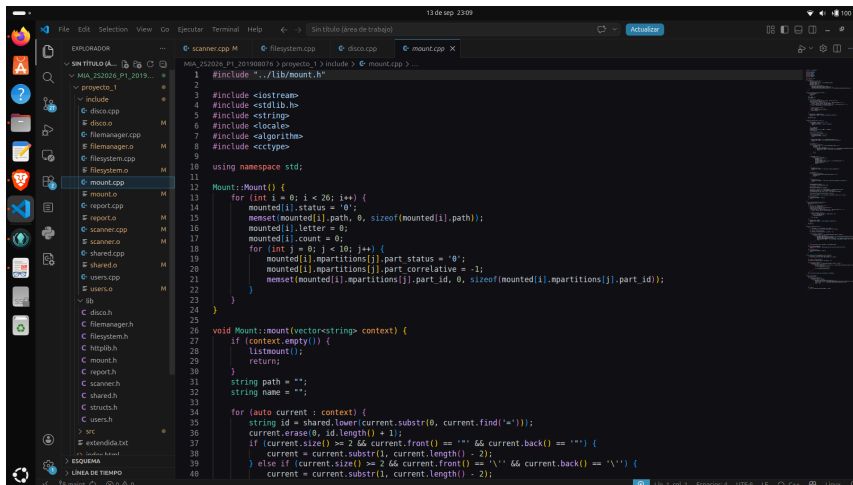
7.7 users.cpp — users.txt



7.8 disco.cpp — mkdisk()



7.9 mount.cpp — ID



8. Compilación y dependencias

El proyecto utiliza C++17. El código fuente se separa entre lib/ (cabeceras) e include/ (implementaciones). La API utiliza httpLib.h como dependencia header-only y los reportes gráficos requieren Graphviz.

El proyecto utiliza C++17. El código fuente se separa entre lib/ (cabeceras) e include/ (implementaciones). La API utiliza httpLib.h como dependencia header-only y los reportes gráficos requieren Graphviz.

Elemento	Uso
Compilador	g++ -std=c++17
Threads	-pthread
Includes	-I./lib
HTTP	httpLib.h
Reportes	Graphviz / dot

9. Tabla de comandos vs. clase

Comando	Clase	Archivo	Método
mkdisk	Disk	disco.cpp	mkdisk()
fdisk	Disk	disco.cpp	fdisk()
rm disk	Disk	disco.cpp	rm disk()
mount	Mount	mount.cpp	mount()
mounted	Mount	mount.cpp	listmount()
mkfs	FileSystem	filesystem.cpp	mkfs()
login	Users	users.cpp	login()
logout	Users	users.cpp	logout()
mkgrp / rmgrp	Users	users.cpp	mkgrp() / rmgrp()
mkusr / rmusr	Users	users.cpp	mkusr() / rmusr()
chgrp	Users	users.cpp	chgrp()
mkdir	FileManager	filemanager.cpp	mkdir()
mkfile	FileManager	filemanager.cpp	mkfile()
cat	FileManager	filemanager.cpp	cat()
rep	Report	report.cpp	rep()

10. Decisiones de diseño

httpLib.h: Se utiliza como librería header-only para exponer HTTP sin añadir una biblioteca binaria separada al núcleo.

main.cpp y api_main.cpp separados: Permite conservar la consola y el servidor REST reutilizando las mismas clases de negocio.

users.txt como archivo real: Permite que usuarios y grupos participen del modelo de almacenamiento, lectura y escritura del EXT2 simulado.

Solo indirecto simple: Mantiene el alcance controlado: 12 apuntadores directos y un Pointerblock de 16 entradas.

char[4] para part_id: El ID documentado es corto, por ejemplo 761A, por lo que un arreglo fijo permite almacenarlo sin memoria dinámica.

11. Limitaciones y alcance

Doble y triple indirecto (i_block[13] e i_block[14]) no implementados.

REMOVE, RENAME, COPY y MOVE no aparecen entre los comandos implementados/documentados del proyecto.

El tamaño máximo documentado de archivo, con 12 directos + 16 indirectos y Fileblock de 64 bytes, es 1,792 bytes.

El manual actual no fija un número máximo de particiones lógicas; estas se organizan mediante la cadena de EBR dentro de la partición extendida.

12. Referencias

Kernel.org — documentación y referencias de sistemas de archivos Linux/EXT2.

Graphviz — documentación del comando dot y sus formatos de salida.

cpp-httpplib — documentación de la librería HTTP header-only.

Enunciado y especificaciones del curso Manejo e Implementación de Archivos.

13. Glosario técnico

Término	Definición
MBR	Master Boot Record; estructura inicial que describe el disco y sus particiones.
EBR	Extended Boot Record; estructura usada para encadenar particiones lógicas.
Inodo	Estructura que representa un archivo o carpeta y sus metadatos/apuntadores.
Bloque	Unidad de almacenamiento para contenido, carpetas o apuntadores.
Bitmap	Mapa de estados que indica recursos libres u ocupados.
Superbloque	Estructura global con contadores, tamaños y offsets de EXT2.
EXT2	Sistema de archivos simulado por el proyecto sobre archivos .mia.
CORS	Mecanismo de headers HTTP para controlar solicitudes entre orígenes.
header-only	Librería integrada mediante cabeceras, sin una biblioteca binaria separada.